

程式設計

Chapter 3 Flow Control

張賢宗

長庚大學人工智慧學系

本投影片修改自書商、林仲志教授之原始版本

Learning Objectives

❖ Branching Mechanisms

- if-else
- switch
- Nesting if-else

❖ Loops

- While, do-while, for
- Nesting loops



Branching Mechanisms

❖ if-else statements

- Choice of two alternate statements based on condition expression
- Example:
if (hrs > 40)
 grossPay = rate*40 + 1.5*rate*(hrs-40);
else
 grossPay = rate*hrs;



widelab
Web Information and Data Engineering Laboratory

if-else Statement Syntax

- ❖ Formal syntax:
if (<boolean_expression>
 <yes_statement>
else
 <no_statement>
- ❖ Note each alternative is only ONE statement!
- ❖ To have multiple statements execute in either branch → use compound statement

Compound/Block Statement

- ❖ Only "get" one statement per branch
- ❖ Must use compound statement { } for multiples
 - Also called a "block" statement
- ❖ Each block should have block statement
 - Even if just one statement
 - Enhances readability
 - 愛用 {} 別人才看得懂
- ❖ 別人看不懂的程式 = 垃圾



widelab
Web Information and Data Engineering Laboratory

Compound Statement in Action

❖ Note indenting in this example:

```
if (myScore > yourScore)
{
```

```
    // 程式記得推縮編排
```

```
    cout << "I win!\n";
    wager = wager + 100;
```

```
}
```

```
else
```

```
{
```

```
    cout << "I wish these were golf scores.\n";
    wager = 0;
```

```
}
```

Common Pitfalls

- ❖ Operator "=" vs. operator "=="
- ❖ One means "assignment" (=)
- ❖ One means "equality" (==)
 - 完全不同的意義 in C/C++!
 - Example:
if (**x = 12**) ← Note operator used!
 Do_Something
else
 Do_Something_Else

The Optional else

❖ else clause is optional

- If, in the false branch (else), you want “nothing” to happen, leave it out. 那可以 if true 什麼都不做嗎？
- Example:
if (sales >= minimum)
 salary = salary + bonus;
cout << "Salary = %" << salary;
- Note: nothing to do for false condition, so there is no else clause!
- Execution continues with cout statement

Nested Statements

❖ if-else statements contain smaller statements

- Compound or simple statements (we' ve seen)
- Can also contain any statement at all, including another if-else stmt!
- Example:
if (speed > 55)
 if (speed > 80)
 cout << "You' re really speeding!" ;
 else //使用最近配對
 cout << "You' re speeding.";
 • Note proper indenting!

Nested Statement

- ❖ 是指一個語句內部包含了另一個或多個語句。
- ❖ 在使用嵌套語句時，應注意確保每一層的邏輯都是清晰和正確的，避免使程式碼過於複雜或引入潛在的錯誤。而且，正確的縮排也是很重要的，它可以增加程式碼的可讀性，使嵌套結構更容易理解。
- ❖ 盡量避免超過三層的嵌套。
- ❖ 可以用函數取代過多的嵌套

Multiway if-else: Display, page 63

- ❖ Not new, just different indenting
- ❖ Avoids "excessive" indenting
 - Syntax:

Multiway if-else Statement

SYNTAX

```
if (Boolean_Expression_1)
    Statement_1
else if (Boolean_Expression_2)
    Statement_2
    .
    .
    .
else if (Boolean_Expression_n)
    Statement_n
else
    Statement_For_All_Other_Possibilities
```

說明

- ❖ **基本結構**：第一個 if 條件用來檢查特定條件 (Boolean_Expression_1)。如果條件成立，執行對應的 Statement_1。
- ❖ **多重條件檢查**：可以使用多個 else if 語句來檢查不同條件 (Boolean_Expression_2 到 Boolean_Expression_n)。當 if 條件不成立時，程式會依序檢查每個 else if 的條件，直到找到成立的條件並執行對應的語句。
- ❖ **最後的 else**：當所有前面的 if 和 else if 條件都不成立時，else 區塊的語句將會執行。這是一個「預設」的分支，用來處理所有未被涵蓋的情況。

Multiway if-else Example: Display, page 63

EXAMPLE

```
if ((temperature < -10) && (day == SUNDAY))  
    cout << "Stay home.";  
else if (temperature < -10) //and day != SUNDAY  
    cout << "Stay home, but call work.";  
else if (temperature <= 0) //and temperature >= -10  
    cout << "Dress warm.";  
else //temperature > 0  
    cout << "Work hard and play hard.";
```

The Boolean expressions are checked in order until the first true Boolean expression is encountered, and then the corresponding statement is executed. If none of the Boolean expressions is true, then the *Statement_For_All_Other_Possibilities* is executed.

Kahoot 一下

❖ <https://play.kahoot.it/v2/?quizId=78206dda-ec33-471a-b022-fb663eae3190>

Self Test

❖ How to write the if-else statement that bas on the following requests?

- (1) `score >= 90, cout << " A grade"`
- (2) `80=< score < 90, cout << "B grade"`
- (3) `60=< score<80 , cout << "C grades"`
- (4) `else, cout<< " D grade"`

可以這樣寫

```
if (score >= 90) {  
    cout << "A grade";  
} else if (score >= 80) {  
    cout << "B grade";  
} else if (score >= 60) {  
    cout << "C grade";  
} else {  
    cout << "D grade";  
}
```

Self Test

- ❖ 輸入人數
- ❖ 輸入年齡
- ❖ 如果
 - 年齡 > 65 , 票價 = 人數 $\times 180$
 - 年齡 < 15 , 票價 = 人數 $\times 120$
 - 其他, 票價 = 人數 $\times 220$

可以這樣寫

```
int people, age;  
int price;
```

```
cout << "請輸入人數: ";  
cin >> people;  
cout << "請輸入年齡: ";  
cin >> age;
```

```
if (age > 65) {  
    price = people * 180;  
} else if (age < 15) {  
    price = people * 120;  
} else {  
    price = people * 220;  
}
```

```
cout << "總票價為: " << price << endl;
```

The switch Statement

- ❖ A new statement for controlling multiple branches
- ❖ Uses controlling expression which returns bool data type (true or false)

switch Statement Syntax:

switch Statement

SYNTAX

```
switch (Controlling_Expression)
{
    case Constant_1:
        Statement_Sequence_1
        break;
    case Constant_2:
        Statement_Sequence_2
        break;
        .
        .
        .
    case Constant_n:
        Statement_Sequence_n
        break;
    default:
        Default_Statement_Sequence
}
```

*You need not place a **break** statement in each case. If you omit a **break**, that case continues until a **break** (or the end of the **switch** statement) is reached.*

The switch Statement in Action: Display, page 64

EXAMPLE

```
int vehicleClass;  
double toll;  
cout << "Enter vehicle class: ";  
cin >> vehicleClass;  
  
switch (vehicleClass)  
{  
    case 1:  
        cout << "Passenger car.";  
        toll = 0.50;  
        break;  
    case 2:  
        cout << "Bus.";  
        toll = 1.50;  
        break;  
    case 3:  
        cout << "Truck.";  
        toll = 2.00;  
        break;  
    default:  
        cout << "Unknown vehicle class!";  
}
```

*If you forget this **break**,
then passenger cars will
pay \$1.50.*



The switch: multiple case labels

- Execution "falls thru" until break
 - switch provides a "point of entry"
 - Example:

```
case 'A' :  
case 'a' :  
    cout << "Excellent: you got an "A"!\n";  
    break;  
case 'B' :  
case 'b' :  
    cout << "Good: you got a "B"!\n";  
    break;
```
 - Note multiple labels provide same "entry"

Char as Integer

在程式設計中，電腦看待字元的方式和我們不同。對我們來說，`'a'` 是一個字母，但對電腦來說，它都是用 `binary` 存放。在 `ASCII` 編碼系統中，每個字元都有一個獨特的數字對應，例如：

`'a'` 的 `ASCII` 值是 97。

`'A'` 的 `ASCII` 值是 65。

所以當我們在程式中寫上 `'a'` 時，電腦實際上是看到 97，它們在電腦裡是同一個東西。這表示我們可以將 `'a'` 當作數字 97 來使用。只有在顯示才會根據不同型態顯示！

switch Pitfalls/Tip

- ❖ Forgetting the break;
 - No compiler error
 - Execution simply "falls thru" other cases until break;
- ❖ Biggest use: MENUs
 - Provides clearer "big-picture" view
 - Shows menu structure effectively
 - Each branch is one menu choice

switch Menu Example

- ❖ Switch stmt "perfect" for menus:

```
switch (response)
{
    case "1":
        // Execute menu option 1
        break;
    case "2":
        // Execute menu option 2
        break;
    case "3":
        // Execute menu option 3
        break;
    default:
        cout << "Please enter valid response.";
}
```

Conditional Operator

❖ Also called "ternary operator"

- Allows embedded conditional in expression
- Essentially "shorthand if-else" operator
- Example:
if ($n1 > n2$)
 $\text{max} = n1$;
else
 $\text{max} = n2$;
- Can be written:
 $\text{max} = (n1 > n2) ? n1 : n2$;
 - "?" and ":" form this "ternary" operator

Loops

❖ 3 Types of loops in C++

- while
 - Most flexible
 - No "restrictions"
- do-while
 - Least flexible
 - Always executes loop body at least once
- for
 - Natural "counting" loop



while Loops Syntax: Display, page 69

Syntax for while and do-while Statements

A while STATEMENT WITH A SINGLE STATEMENT BODY

```
while (Boolean_Expression)  
    Statement
```

A while STATEMENT WITH A MULTISTatement BODY

```
while (Boolean_Expression)  
{  
    Statement_1  
    Statement_2  
    .  
    .  
    .  
    Statement_Last  
}
```

while Loop Example

❖ Consider:

```
count = 0;  
while (count < 3)  
{  
    cout << "Hi ";  
    count++;  
    expression  
}
```

```
// Initialization  
// Loop Condition  
  
// Loop Body  
// Update
```

- Loop body executes how many times?

do-while Loop Syntax: Display, page 70

A do-while STATEMENT WITH A SINGLE-STATEMENT BODY

```
do  
    Statement  
while (Boolean_Expression);
```

A do-while STATEMENT WITH A MULTISTatement BODY

```
do  
{  
    Statement_1  
    Statement_2  
    .  
    .  
    .  
    Statement_Last  
} while (Boolean_Expression);
```

*Do not forget
the final
semicolon.*

do-while Loop Example

```
❖ count = 0;           // Initialization
do
{
    cout << "Hi ";      // Loop Body
    count++;            // Update expression
} while (count < 3);    // Loop Condition
```

- Loop body executes how many times?
- do-while loops always execute body at least once!

while vs. do-while

- ❖ Very similar, but...
 - One important difference
 - Issue is "WHEN" boolean expression is checked
 - while: checks BEFORE body is executed
 - do-while: checked AFTER body is executed
- ❖ After this difference, they' re essentially identical!
- ❖ while is more common, due to it' s ultimate "flexibility"

Comma Operator

- ❖ Evaluate list of expressions, returning value of the last expression
- ❖ Most often used in a for-loop
- ❖ Example:
first = (first = 2, second = first + 1);
 - first gets assigned the value 3
 - second gets assigned the value 3
- ❖ No guarantee what order expressions will be evaluated.

for Loop Syntax

for (Init_Action; Bool_Exp; Update_Action)
 Body_Statement

- ❖ Like if-else, Body_Statement can be a block statement
 - Much more typical

for Loop Example

- ❖

```
for (count=0;count<3;count++)  
{  
    cout << "Hi ";    // Loop Body  
}
```
- ❖ How many times does loop body execute?
- ❖ Initialization, loop condition and update all "built into" the for-loop structure!
- ❖ A natural "counting" loop

Loop Issues

❖ Loop's condition expression can be ANY boolean expression

❖ Examples:

```
while (count<3 && done!=0)
```

```
{
```

```
    // Do something
```

```
}
```

```
for (index=0;index<10 && entry!=-99)
```

```
{
```

```
    // Do something
```

```
}
```

Loop Pitfalls: Misplaced ;

❖ Watch the misplaced ; (semicolon)

- Example:

```
while (response != 0) ;  
{  
    cout << "Enter val: ";  
    cin >> response;  
}
```

- Notice the ";" after the while condition!

❖ Result here: INFINITE LOOP!

Loop Pitfalls: Infinite Loops

- ❖ Loop condition must evaluate to false at some iteration through loop
 - If not → infinite loop.
 - Example:

```
while (1)
{
    cout << "Hello ";
}
```
 - A perfectly legal C++ loop → always infinite!
- ❖ Infinite loops can be desirable
 - e.g., "Embedded Systems"

The break and continue Statements

❖ Flow of Control

- Recall how loops provide "graceful" and clear flow of control in and out
- In RARE instances, can alter natural flow

❖ break;

- Forces loop to exit immediately.

❖ continue;

- Skips rest of loop body

❖ These statements violate natural flow

- Only used when necessary!

Nested Loops

- ❖ Recall: ANY valid C++ statements can be inside body of loop
- ❖ This includes additional loop statements!
 - Called "nested loops"
- ❖ Requires careful indenting:

```
for (outer=0; outer<5; outer++)  
    for (inner=7; inner>2; inner--)  
        cout << outer << inner;
```

 - Notice no { } since each body is one statement
 - Good style dictates we use { } anyway

Self test

- ❖ 輸入5科成績,計算後輸出總分與平均
 - 輸出判斷
 - 平均大於90輸出" A"
 - 平均小於60輸出" C"
 - 其他輸出" B"
- ❖ 1-100的迴圈裡,將5的倍數加總起來,請問加幾次之後這個總和超過50

❖ 利用switch case完成下列程式

- 當輸入 "a" or "A", 請輸出 "Apple"
- 當輸入 "b" or "B", 請輸出 "Boy"
- 其他輸入, 則輸出 "Welcome!!"

Self Test

- ❖ 輸入N找出介於1到N的質數
- ❖ 利用while loop 寫出九九乘法表
- ❖ 利用do ... while loop 寫出九九乘法表
- ❖ 輸入N 利用for loop 印出以下圖形
- ❖ 輸入N利用for loop 印出

```
*
**
***
****
***
**
*

*
**
***
****
*****
```

Self test

- ❖ 輸入5個數字,找出最大值與最小值
- ❖ 輸入攝氏溫度C,計算華氏溫度F
 - $F = (9/5) * C + 32$

Self test

❖ 計算周長與面積

- 輸入四邊形邊長,計算出周長與面積
- 輸入正三角形底與高,計算周長與面積
- 輸入半徑,計算圓的周長與面積

❖ 利用switch case

- 當輸入" 1" ,讓使用者輸入四邊形邊長,計算周長與面積
- 當輸入" 2" ,輸入正三角形底與高,計算周長與面積
- 當輸入" 3" ,輸入半徑,計算圓的周長與面積

Q : 如果要讓程式可以一直執行(讓使用者不斷選擇想要的計算)直到使用者按下"E" or "e"才會脫離程式,本程式該如何修改?

Self test

❖ 撰寫一個程式可以讓使用者不斷輸入整數,當是正數時可以繼續輸入下一個數字,當出現負數時,則程式終結,輸出” It is negative number!!”

Self test

❖ What is the output of the following?

- `Int count = 3;`
`while(--count > 0)`
`cout << count << " ";`
- `Int n = 1;`
`do`
`cout << n << " ";`
`while( n++ <= 3);`
- `Int n = 1;`
`do`
`cout << n << " ";`
`while( ++n <= 3);`

Self Test

```
❖ int i = 1;  
  while( i <= 10)  
  {  
    if( i < 5 && i != 2)  
      cout << 'X' ;  
    i++;  
  }
```

Self Test

```
❖ int i = 1;  
  while( i <= 10)  
  {  
    cout << 'X' ;  
    i = i + 3;  
  }
```

Self Test

- ❖ 請利用For迴圈，利用if 判斷並印出1~100間的偶數。
- ❖ 請利用For迴圈，不能if 判斷並印出1~100間的偶數。

Summary 1

- ❖ C++ branching statements
 - if-else, switch
 - switch statement great for menus
- ❖ C++ loop statements
 - while
 - do-while
 - for

Summary 2

- ❖ do-while loops
 - Always execute their loop body at least once
- ❖ for-loop
 - A natural "counting" loop
- ❖ Loops can be exited early
 - break statement
 - continue statement
 - Usage restricted for style purposes